



第四届eBPF开发者大会

The 4th eBPF Developer Conference (eBPFDC 2026)

陕西·西安

2026年4月18日

<https://www.ebpftravel.com>

sysTrace：基于eBPF的AI全栈性能观测方案

分享人：李蔚

目录

- 智算集群性能诊断的挑战
- 技术架构与关键能力
- 效果展示
- 未来展望

智算集群性能诊断的挑战

智算集群性能诊断的挑战

大模型训练是一个涉及超大规模计算、海量数据交互、复杂分布式协同的系统工程。训练过程中的性能瓶颈与稳定性问题，直接决定了模型的训练效率、研发成本和最终上线效果。



训练周期漫长

千亿参数大模型的训练往往持续数周甚至数月，任何微小的性能抖动都可能导致整体周期大幅延长。



资源成本高昂

因算力资源利用不均或性能阻塞导致的资源浪费，最高可达训练总成本的 20%~30%，经济损失显著。



性能诊断核心目标

构建全链路、高实时的性能监测体系，打破“黑盒”训练状态，实现问题的分钟级发现与精准定位。

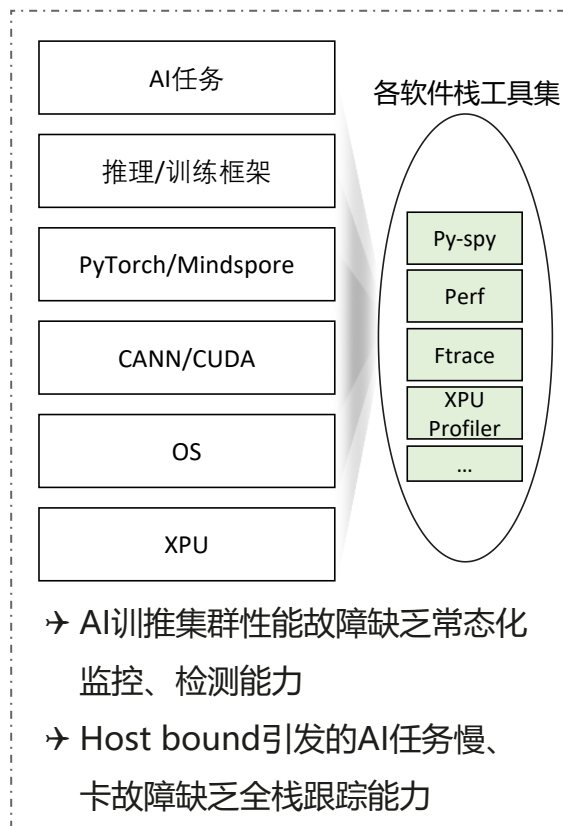
核心痛点：如何在复杂分布式环境中，及时检测并迅速定位性能问题，是保障大模型稳定高效训练的关键。



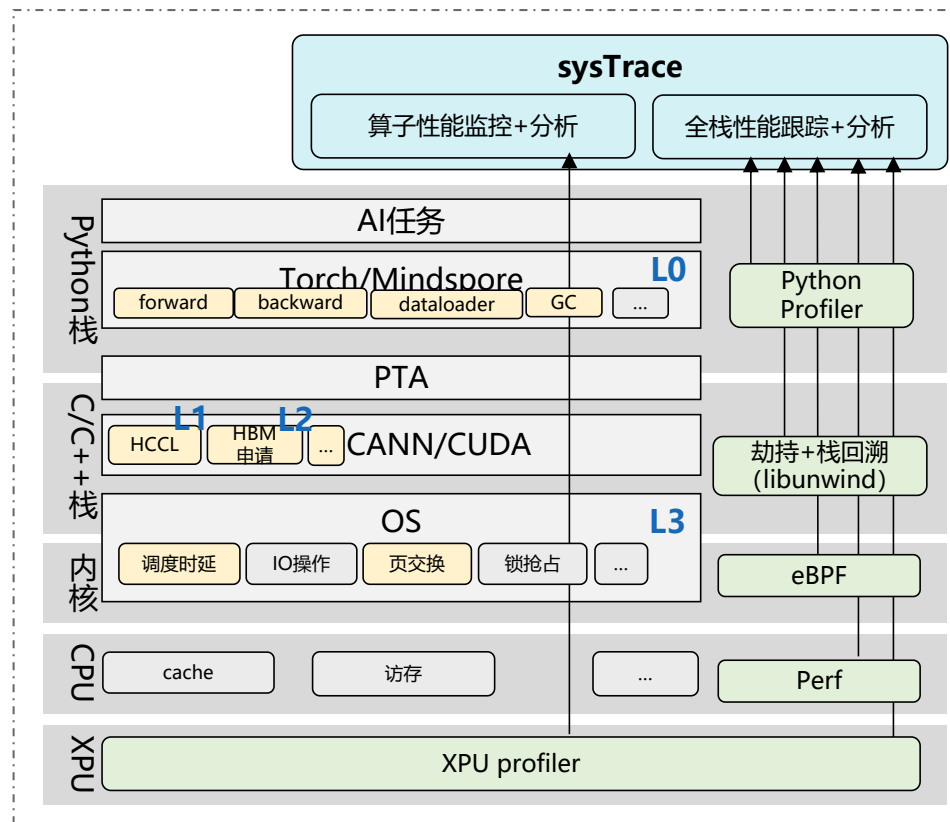
技术架构和关键能力

从“逐层孤立分析”到“全栈协同分析”

AS IS: 逐层孤立分析



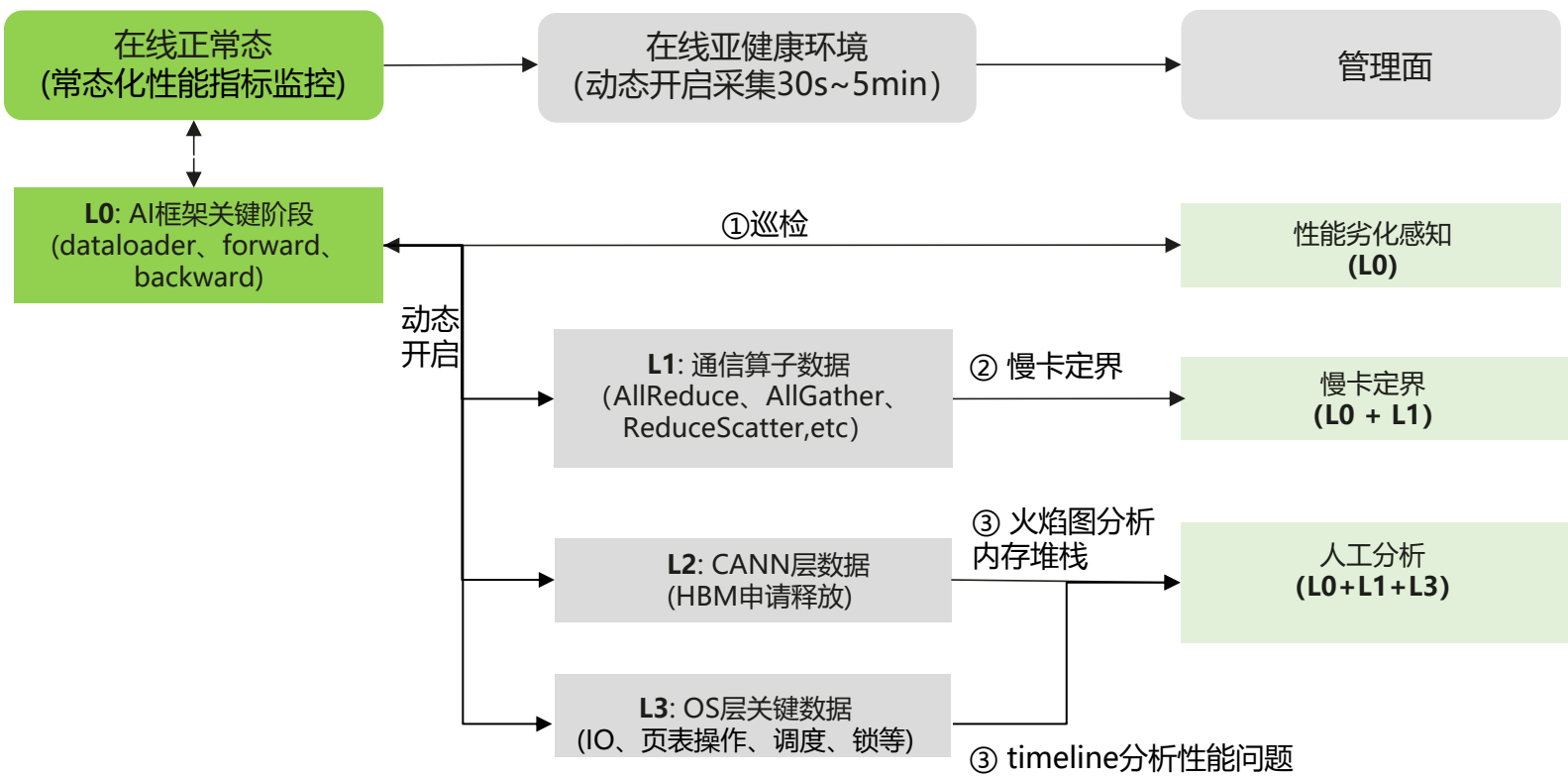
TO BE: 全栈协同分析



通过构建全栈协同分析体系，打破数据孤岛。将复杂的性能故障定位时间从传统的“数天级”缩短至“小时级”，全面提升系统的稳定性与运维效率。

sysTrace关键能力：动态分层采集

性能故障在线诊断



关键步骤

- ①**巡检**: 周期性运行性能劣化感知当前训练任务是否有性能问题，输入类step时延数据，输出是否劣化，劣化故障时刻
- ②**慢卡定界**: 性能劣化后，开启慢卡定界，输入通信算子数据(5分钟左右)，输出故障节点，时间，覆盖计算慢、通信慢、算子下发慢、IO慢问题；
- ③**性能定位**: 以火焰图和timeline形式展现pytorch层、CANN层和OS层数据，联合分析性能问题

核心价值：“按需采集”模式在保证问题诊断深度的同时，降低监控带来的性能开销，实现大模型训练的高效稳定运行。

eBPF在AI性能观测中的应用



CPU调度分析

通过 `sched\_switch` 等tracepoint，精确追踪AI线程的CPU占用、调度延迟与运行队列长度。识别非核心线程（如数据加载）抢占资源的瓶颈，从而优化CPU亲和性与任务优先级。

💡 场景示例

识别数据加载与模型计算线程的CPU资源竞争，调整优先级后显著降低端到端时延、提升整体训练吞吐量。



锁争用分析

利用 `uprobe` 动态探测同步原语，量化锁的等待时间与阻塞线程。诊断PyTorch等框架中因GIL或自定义锁导致的同步瓶颈，定位性能下降的根源。

💡 场景示例

定位数据预处理阶段的锁竞争瓶颈，通过优化锁的粒度或引入无锁数据结构，解决性能卡顿问题。



内存性能分析

捕获 `page\_fault` 等内核事件，分析缺页率与内存交换活动。诊断内存泄漏、访问效率低下及OOM风险，追踪算子或数据结构的异常内存申请行为。

💡 场景示例

发现训练任务在保存CKPT阶段出现性能爬坡现象，定位出该阶段由于开启了透明大页特性，频繁出现内存回收和规整，导致性能下降。

核心价值：通过对CPU、锁、内存等核心系统资源的深度剖析，eBPF能够精准定位AI应用的性能瓶颈，实现从系统底层到应用层的端到端性能优化。

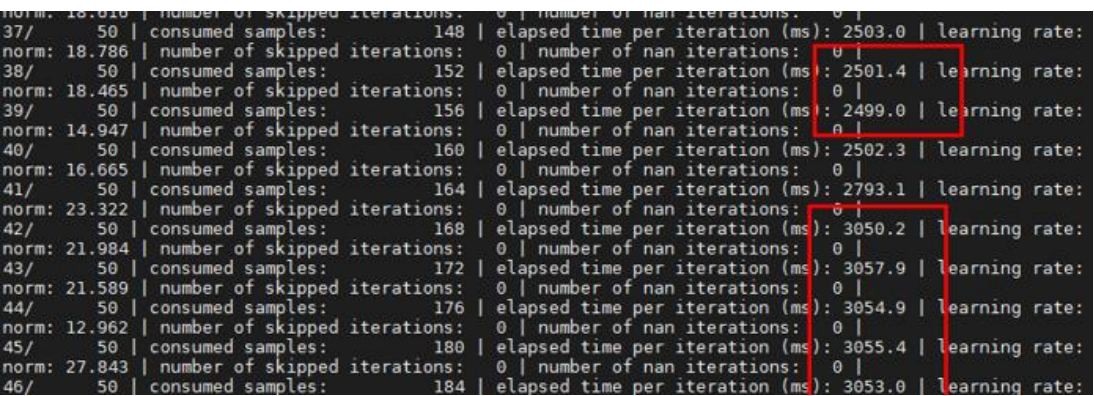
效果展示

采集数据展示---NPU采集

注入AICore占用

● 模型配置DP1-TP4-PP2

注入故障方式：在3卡上启动小模型占用，导致主任务训练性能下降

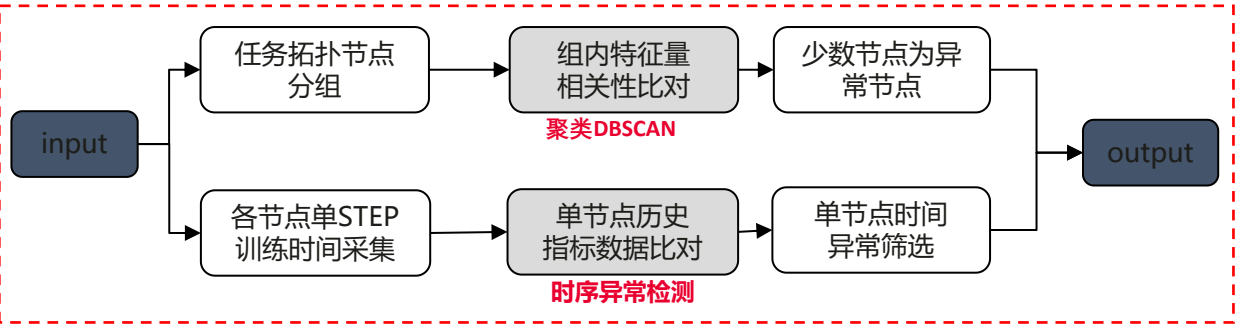


Step时延 baseline (ms)	Step时延 故障时刻均值 (ms)	Step时延 增长率
2501.4	3054.4	22.1%

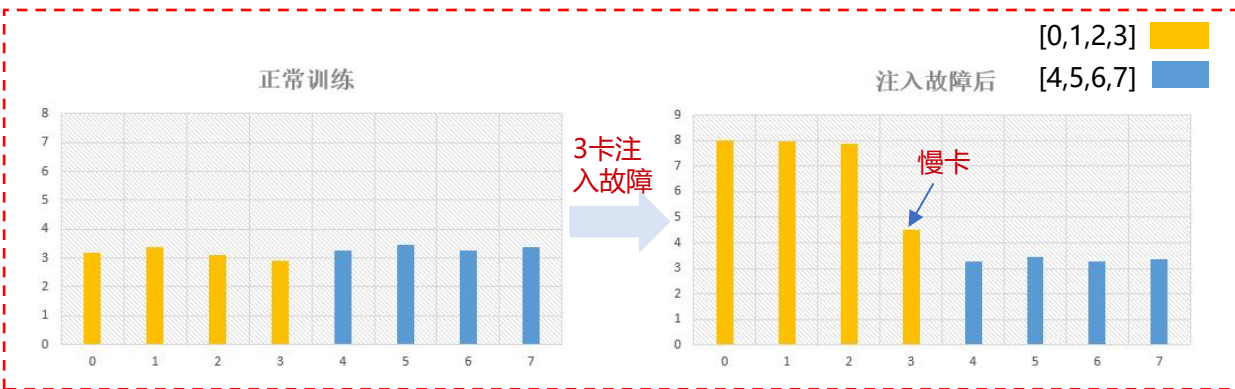
- 1. 正常训练时，每个step平均耗时2501.4ms；发生性能劣化时，step耗时增加至3054.4ms，**同比增长22.1%**
- 2. 以模型训练需200000个step为例：
 - 正常训练集群需耗时5.78天；
 - 当集群存在慢节点，导致性能劣化，训练完成需7.06天；**导致1.68天的单机训练资源浪费**

慢节点定界

- 方案：八卡分按照PP粒度分为两组，[0,1,2,3]和[4,5,6,7]



- TP组 hcclAllGather对比检测结果：



● 结论：

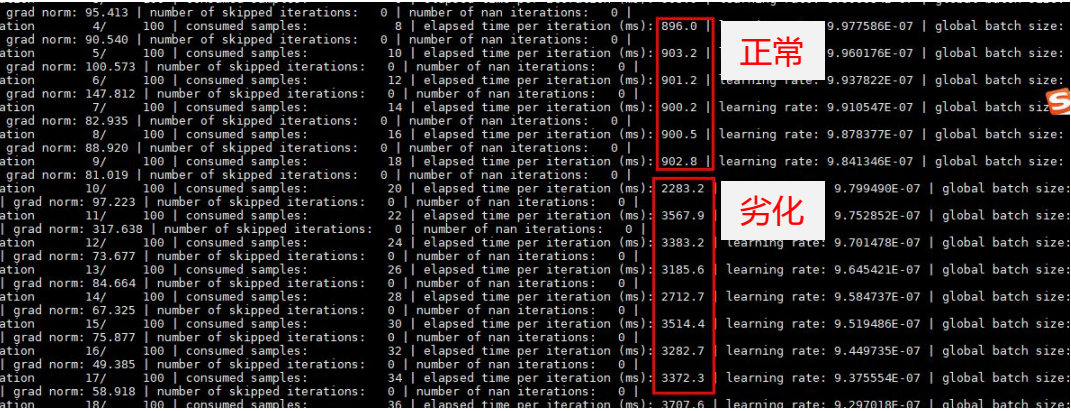
- ✓ 计算慢会导致3卡的计算时间明显增加，同一组内的其他卡等待，导致通信算子时延增加，所以组内时延不变的卡确定为慢卡

采集数据展示---内存采集

注入内存压力的故障

● 注入方式:

- 1) 在训练脚本中加入numa绑核命令:
numactl --membind=1 python -m torch.distributed.launch
- 2) 对同一个numa节点注入内存压力:
numactl --membind=1 stress-ng --vm 4 --vm-bytes \$(awk '/MemAvailable/{printf "%dM", \$2 / 1024 * 0.99;}'/proc/meminfo) --timeout 60s --stream 600



● 时延统计: 计算连续5个step时延值取平均

Step时延 baseline (ms)	Step时延 故障时刻均值 (ms)	Step时延 增长率
901.58	3,026.52	235.7%

1. 正常训练时, 每个step平均耗时901.58ms; 发生性能劣化时, step耗时增加至3026.52ms, 同比增长235.7%

定界思路

- 方案: 生成系统关键事件的timeline, 每个rank以线程为粒度展示
- 异常卡的mm_fault时延比正常卡mm_fault的时延明显增长(1569us -> 889327us), 统计数据的分布直方图辅助分析, 注入压力后数据分布更分散, 用信息熵作为判据



● 信息熵对比, 可设置2作为阈值

rank	0	1	2	3	4	5	6	7
正常	1.6202	1.6808	1.6717	1.6798	1.7336	1.6439	1.6398	1.5726
异常	2.3738	2.0543	2.3503	2.2179	2.0522	2.2815	2.0075	2.2456

采集数据展示---CPU调度

注入CPU压力的故障

- 注入方式: stress-ng --cpu 0 --cpu-method matrixprod --timeout 60s

grad norm: 44.337	number of skipped iterations: 0	number of nan iterations: 0	elapsed time per iteration (ms): 2558.2	learning rate
24/ 2000	consumed samples: 96			
grad norm: 35.545	number of skipped iterations: 0	number of nan iterations: 0	elapsed time per iteration (ms): 2550.9	learning rate
25/ 2000	consumed samples: 100			
grad norm: 37.717	number of skipped iterations: 0	number of nan iterations: 0	elapsed time per iteration (ms): 2554.7	learning rate
26/ 2000	consumed samples: 104			
grad norm: 38.698	number of skipped iterations: 0	number of nan iterations: 0	elapsed time per iteration (ms): 2573.1	learning rate
27/ 2000	consumed samples: 108			
grad norm: 33.996	number of skipped iterations: 0	number of nan iterations: 0	elapsed time per iteration (ms): 2557.7	learning rate
28/ 2000	consumed samples: 112			
grad norm: 22.248	number of skipped iterations: 0	number of nan iterations: 0	elapsed time per iteration (ms): 2500.5	learning rate
29/ 2000	consumed samples: 116			
grad norm: 27.730	number of skipped iterations: 0	number of nan iterations: 0	elapsed time per iteration (ms): 2868.6	learning rate
30/ 2000	consumed samples: 120			
grad norm: 32.014	number of skipped iterations: 0	number of nan iterations: 0	elapsed time per iteration (ms): 2708.7	learning rate
31/ 2000	consumed samples: 124			
grad norm: 85.323	number of skipped iterations: 0	number of nan iterations: 0	elapsed time per iteration (ms): 2695.1	learning rate
32/ 2000	consumed samples: 128			
grad norm: 26.797	number of skipped iterations: 0	number of nan iterations: 0	elapsed time per iteration (ms): 2904.5	learning rate
33/ 2000	consumed samples: 132			
grad norm: 31.935	number of skipped iterations: 0	number of nan iterations: 0	elapsed time per iteration (ms): 2961.2	learning rate
34/ 2000	consumed samples: 136			

- 时延统计: 计算连续5个step时延值取平均

Step时延 baseline (ms)	Step时延 故障时刻均值 (ms)	Step时延 增长率
2550.4	2825.6	10.8%

- 正常训练时, 每个step平均耗时2550.4ms; 发生性能劣化时, step耗时增加至2825.6ms, 同比增长10.8%

CPU压力故障分析

- 方案: 生成系统关键事件的timeline, 每个rank以线程为粒度(python主线程、ACL线程)展示



未来展望

特性一：刻画业务拓扑画像

在AI应用，特别是大语言模型（LLM）推理服务的部署与运维中，我们正面临前所未有的复杂性挑战。异构硬件混合、Agent多层嵌套等问题，使得传统观测手段难以穿透系统内核，导致内部状态成为难以窥探的“黑盒”。

背景必要性：从“盲人摸象”到“全景导航”的迫切需求

- **系统复杂性高**：在当今复杂的AI系统中，业务逻辑日益庞大，一个典型的AI任务往往涉及数十甚至上百个协同工作的进程与线程。
- **开发者视角局限**：开发者通常只能监控自己负责的模块或单个进程，无法看到整个系统的全貌，难以理解进程间的依赖关系和资源争抢情况。
- **问题定位困难**：当性能出现瓶颈时，很难判断是自身模块的问题，还是被其他进程“干扰”所致。这种“看不清、看不全、看不透”的现状，使得性能调优工作如同“盲人摸象”。

eBPF核心实现思路：基于eBPF的动态拓扑构建

- **eBPF探针无侵入数据采集**：在内核层面部署eBPF探针，Hook住进程/线程创建与退出、网络连接、文件IO等关键系统调用，实时捕获所有进程和线程的生命周期事件、通信行为和资源交互。
- **动态构建业务拓扑关系图谱**：将采集到的原始事件流进行实时处理和关联分析，动态构建一个包含进程、线程、容器、节点等多维度的拓扑关系图谱。
- **可视化呈现与智能分析**：将构建好的拓扑图谱通过可视化界面进行展示，并结合AI算法进行分析，自动识别关键路径、异常依赖和潜在的性能瓶颈。



特性二：观测Agent的调用链

当前对AI Agent的观测普遍停留在请求次数、总时延等宏观结果指标上，无法揭示其内部复杂的工具调用逻辑，导致Agent行为如同黑盒。我们需要超越浅层指标，通过观测完整的调用链，实现对Agent执行过程的深度洞察。

背景必要性：打破Agent的黑盒困境

- **逻辑复杂性**：任务涉及多次工具调用，但中间的调用详情、耗时、IO均不可见。
- **观测局限性**：宏观指标与内部操作割裂，无法定位任务延迟是源于LLM还是工具调用。

核心价值：迈向“Harness Engineering”

- **过程洞察**：清晰呈现任务全链路，为调试优化提供深度依据。
- **可解释性**：失败时快速定位问题节点，增强系统的鲁棒性与可信度。

eBPF 核心实现思路：构建“意图-行为”因果链

步骤一：系统捕获

部署eBPF探针，追踪Agent主进程及其所有子进程的系统调用（如execve, connect），捕获Agent的系统级行为。

步骤二：因果关联

设计关联引擎，将LLM的输出（意图）与后续的系统调用（行为）进行匹配，建立意图与行为的因果关联。

步骤三：链路可视化

将关联起来的“意图-行为”序列整合成一个完整的分布式追踪Span，并输出到标准的可观测性后端进行可视化分析。



THANKS